NutriResQ

Smart Food Rescue & Redistribution Optimizer

Detailed Project Documentation

CT-363 Design and Analysis of Algorithms (DAA) — Complex Computing Problem

Instructor: Tooba Shaikh

Department of CSIT, NED University of Engineering and Technology, Karachi

Group Name: Codetz

Team Member	Roll No.
Muhammad Umar (Team Lead)	**CT-24028**
Muhammad Ibtissam Aslam	**CT-24020**
Muhammad Ahmed Khan	**CT-24041**
Syed Ahmed Ayaan	**CT-24048**

Planning baseline — October 2026

Table of Contents

1. Introduction and Project Overview . . . 3
2. The Real-World Problem . . . 4
3. Users and Participants . . . 5
4. Product Requirements . . . 5
5. The Four Algorithms in Detail . . . 10
6. Data, Constraints and Modeling Rules . . . 13
7. The Three Main Interactive Features . . . 15
8. System Workflow and Optimization Pipeline . . . 16
9. System Architecture . . . 18
10. Technology Stack . . . 21
11. Database Design . . . 23
12. API Design . . . 30
13. Use Case Model . . . 31
14. Project Structure . . . 33
15. Testing and Evaluation Plan . . . 33
16. Mapping to the DAA CCP Requirements . . . 34
17. Roadmap, Scope and Governance . . . 34

1 Introduction and Project Overview

NutriResQ is the CT-363 **Design and Analysis of Algorithms (DAA)** course project (Complex Computing Problem) and also a full-stack portfolio project. It is an algorithm-driven **decision-support system for planning surplus-food collection and redistribution**. It combines algorithm design, a full-stack web application, interactive maps, optimization experiments and scenario simulation in one reproducible workflow.

The goal is **not** to build a website where restaurants list surplus food. The goal is to build a system that helps an operations coordinator explore a controlled food-rescue network, allocate food packages to vehicles with limited capacity, identify geographically close locations, plan routes over an available-road network, compare algorithms with simpler baselines, and test what happens when conditions change.

1.1 Questions the system answers

- Which available roads should the system use to connect food donors, collection hubs and recipient organizations?
- How should limited vehicle capacity be allocated across food packages to maximize their humanitarian impact?
- Which locations are geographically closest and potentially suitable for collection or delivery?
- What delivery-stop order produces a low-cost route?
- What happens when a road closes, a vehicle's capacity changes, or an important recipient is removed?
- How much better does one algorithm perform than a simpler baseline?

Note: The flood-relief system in the original assignment brief is the *reference scenario* for its algorithmic requirements, not the application being built. NutriResQ adapts the underlying optimization concepts to food rescue.

1.2 Project at a glance

Item	Detail
Project name	NutriResQ (originally named NutriVentra)
Full title	NutriResQ — Smart Food Rescue & Redistribution Optimizer
Course	CT-363 Design and Analysis of Algorithms (DAA), Complex Computing Problem (CCP)
Project type	Academic DAA CCP and full-stack portfolio project
Primary user	Operations Coordinator
Initial data	Controlled, documented synthetic dataset (no real donors, charities or live inventories)
Required algorithms	A* Search, Multiple Knapsack (Dynamic Programming), Closest Pair of Points (Divide and Conquer), Ant Colony Optimization
Main interactive features	Optimization Dashboard, Algorithm Comparison Laboratory, Scenario Simulator
Technology	React, TypeScript, Tailwind CSS, Leaflet + OpenStreetMap, Recharts, Node.js + Express, plain JavaScript algorithms, PostgreSQL on Neon
Deployment	Vercel (frontend), Render or compatible Node.js host (API), Neon (database)
Status	Planning / pre-implementation
Guiding principle	Correct, explainable optimization first; polished interface second

1.3 The four algorithms at a glance

Algorithm	Course paradigm	Responsibility in NutriResQ
A* Search	Greedy / search	Finds a low-cost road path between two locations over available roads
Multiple Knapsack	Dynamic Programming	Assigns eligible food packages to several vehicles within capacity to maximize total impact
Closest Pair of Points	Divide and Conquer	Finds the geographically closest pair among eligible locations
Ant Colony Optimization	Approximation / heuristic	Finds a low-cost visiting order for a vehicle's multi-stop route

The algorithms solve *different* subproblems. A* computes a road path between two endpoints; ACO proposes a multi-stop order. ACO must never be described as replacing A*. This distinction is what prevents the project from becoming four unrelated demonstrations.

2 The Real-World Problem

Food donors such as restaurants, bakeries and supermarkets often have surplus food, while recipient organizations such as shelters, welfare organizations, community kitchens and food banks have needs. Coordinating these resources is difficult. Food rescue networks face four connected challenges.

Problem	What goes wrong	What the system must do
1. Inefficient road connectivity	Donors and recipients are spread across locations; some roads are unavailable and travel costs differ.	Understand which locations can be reached and how to navigate the available network.
2. Limited vehicle capacity	Available food can exceed the capacity of the delivery fleet, so sending everything is impossible.	Select and distribute packages to maximize impact without exceeding vehicle limits.
3. Expensive delivery routes	Visiting donors and recipients in a poor order wastes distance, time and vehicle resources.	Construct and improve multi-stop delivery routes.
4. Changing conditions	Roads close, donations expire, recipient demand changes and vehicles become unavailable.	Allow conditions to be changed and results to be recalculated.

2.1 Main objective

NutriResQ aims to **maximize the humanitarian value of food redistributed** while respecting vehicle capacities, food eligibility, road availability and delivery feasibility, while also **reducing unnecessary travel**.

> **Note:** These objectives are related but not identical. Maximizing food allocation does not automatically minimize delivery distance. NutriResQ therefore uses multiple algorithms for different subproblems and integrates their outputs into one decision-support workflow. The UI and documentation must make this trade-off visible.

2.2 What NutriResQ helps the coordinator do

- understand which locations are connected through available roads;
- identify unreachable locations and avoid routes that use closed roads;
- select and allocate food packages within vehicle capacity limits;
- inspect geographic proximity between locations;
- build and compare multi-stop routes;
- change a scenario and rerun optimization;
- compare algorithm outputs against simpler baselines; and
- inspect or reproduce prior optimization runs.

2.3 Why a synthetic dataset

For the CCP, the initial implementation uses a controlled synthetic dataset. That makes results reproducible without pretending the application is connected to real restaurants, charities or live donation inventories. The system must not imply that real donors, live traffic or live inventories are connected unless those integrations are explicitly implemented later.

3 Users and Participants

The system models four principal types of participants. Only the Operations Coordinator is an interactive user in the MVP; the others are records in the database.

Participant	Description	Initial representation
Food donors	Restaurants, bakeries, supermarkets and other participating businesses. Each has a location and may have one or more available food batches.	Record with a location and one or more donation/package records
Recipient organizations	Welfare organizations, shelters, community kitchens and food banks. Each has a location, food requirements and an urgency/priority score.	Record with a location, demand/priority information and availability
Delivery fleet	Vehicles with individual capacity limits, availability and assigned delivery work.	Record with capacity and availability
Operations Coordinator	The primary application user. Manages the scenario, runs optimization, inspects results, compares algorithms and simulates changes.	The only required interactive user role for the MVP

3.1 Primary persona: Operations Coordinator

Needs: understand the current scenario; edit roads, packages, recipient priorities and vehicle capacities; run optimization and understand its output; identify unreachable locations and infeasible packages; compare algorithms and baselines; simulate a disruption; and revisit an earlier run.

Success looks like: the coordinator can produce and explain a valid plan without manually calculating every route or capacity assignment.

3.2 Authentication and portals

Authentication and separate donor/recipient portals are optional extensions, not prerequisites for proving the algorithms work. Donors, recipients and vehicles are not login roles in the MVP. Possible future actors are a donor representative, a recipient representative and a driver, if corresponding portals are ever added.

4 Product Requirements

4.1 Product vision

Build a usable, explainable food-rescue optimization tool that combines algorithmic analysis with an interactive full-stack application. A coordinator should be able to model a small food-rescue network, optimize the use of limited fleet capacity, inspect routes, compare algorithms and reproduce results after scenario changes.

Product promise: turn a scenario's road, donation, recipient and fleet data into feasible, inspectable optimization results. The system is a decision-support prototype for academic evaluation and portfolio demonstration. It does not autonomously dispatch real drivers or certify food safety.

4.2 Goals

- Implement four required algorithms as independent, testable modules.
- Integrate their results into a coherent optimization workflow.
- Enforce capacity, eligibility and road-availability constraints.
- Report infeasible, unreachable or partial results instead of concealing them.

- Compare selected algorithms with transparent baselines.
- Allow controlled scenario changes and reruns.
- Persist scenario snapshots and results in PostgreSQL.
- Provide a clear dashboard, map, allocation table, route view and benchmark comparison.
- Support an academic demonstration with measured results and documented complexity.

4.3 Non-goals for the MVP

- production logistics management for real organizations;
- live maps-based traffic, real-time ETAs or guaranteed road routing;
- real-time GPS/driver dispatch;
- donor and recipient self-service portals;
- native mobile applications;
- AI/ML demand prediction;
- payments, invoicing or a donation marketplace;
- comprehensive food-safety compliance workflows;
- multi-tenant enterprise access control; and
- claims that the heuristic (ACO) route is globally optimal.

4.4 Core user journeys

Journey	Steps
A — Inspect a scenario	Open the application → select a saved scenario or create a new one → inspect locations, road links, donations, recipients and vehicles → see validation errors and warnings before running.
B — Run integrated optimization	Select a valid scenario → start a run → view reachability and A* paths → view closest-pair analysis → view allocation by vehicle with unallocated reasons → view each vehicle's stop order, road paths and cost → review metrics and warnings → save and revisit the run.
C — Compare algorithms	Open the comparison area → select an algorithm and its baseline → run both on the same input → view measured quality and runtime → see metric definitions, limitations and test settings.
D — Simulate a disruption	Select a scenario or prior run → apply a change (close a road, reduce capacity, reduce donations, expire a donation, make a recipient unavailable) → original stays unchanged → rerun → compare before and after.
E — Review history	Open prior runs → inspect snapshot, algorithm configuration, status, metrics, allocations and routes → optionally rerun from a copied scenario without mutating the historical result.

4.5 Functional requirements

Priority: **P0** = required for a credible MVP; **P1** = important after the core workflow works; **P2** = future extension.

Scenario and data management

ID	Pri.	Requirement
FR-01	P0	Load a seeded synthetic scenario on first setup.
FR-02	P0	List and select available scenarios.
FR-03	P0	Create a scenario by copying a known valid dataset or starting from the documented template.
FR-04	P0	Edit scenario-specific road availability, vehicle capacity/availability, donation eligibility/quantity and recipient availability/priority where supported.
FR-05	P0	Validate required fields, references, non-negative quantities and supported value ranges before optimization.
FR-06	P0	Show validation errors that identify the offending record and field.
FR-07	P0	Preserve a prior run's input snapshot even if the scenario is later edited.

A* road pathfinding

ID	Pri.	Requirement
FR-08	P0	Represent roads as a weighted graph with location nodes and directed or explicitly modeled undirected edges.
FR-09	P0	Exclude closed/unavailable roads from path calculations.
FR-10	P0	Return a path and total cost when a route exists; return an explicit unreachable result otherwise.
FR-11	P0	Use an admissible heuristic for the selected edge-cost model, or a zero heuristic if no safe heuristic is available.
FR-12	P1	Compare path cost and runtime against Dijkstra using identical graph inputs.
FR-13	P1	Support a road-failure experiment that closes three connections and compares full recomputation with a clearly defined affected-path repair strategy. Do not label this an incremental minimum-spanning-tree algorithm.

Multiple-knapsack dynamic programming

ID	Pri.	Requirement
FR-14	P0	Allocate each eligible package to at most one vehicle.
FR-15	P0	Never exceed any vehicle's capacity.
FR-16	P0	Maximize the sum of the defined package values/priorities within the chosen model.
FR-17	P0	Reconstruct and display the selected packages for each vehicle.
FR-18	P0	Report expired/ineligible packages and packages that individually exceed every available vehicle's capacity.
FR-19	P0	Return the best feasible partial allocation if not all packages can be assigned; do not fabricate full fulfillment.
FR-20	P1	Compare against a documented greedy baseline using the same input and objective.
FR-21	P0	Validate the DP state size and input limits before running; return a clear error for unsupported sizes rather than freezing the server.

Closest pair of points

ID	Pri.	Requirement
FR-22	P0	Find the closest pair among configured eligible locations using divide and conquer.
FR-23	P0	State the coordinate/distance model and units in the result.
FR-24	P0	Handle duplicate coordinates and fewer than two eligible points.
FR-25	P1	Compare with a brute-force baseline on identical points.
FR-26	P0	Do not present straight-line geographic proximity as a road travel distance.

ACO route optimization

ID	Pri.	Requirement
FR-27	P0	Build a travel-cost matrix for required stops using A* path costs over available roads.
FR-28	P0	Use ACO to search for a low-cost ordering of the stops in a vehicle's feasible route.
FR-29	P0	Validate every route leg against the road graph and calculate the displayed cost from the same cost source.
FR-30	P0	Handle disconnected stops explicitly; never silently assign an infinite-cost leg to a displayed route.
FR-31	P0	Label ACO output as a heuristic result, not a guaranteed global optimum.
FR-32	P1	Compare with a nearest-neighbor baseline.
FR-33	P1	Support fixed seeds and repeated trials where practical; display the trial settings.

Integrated optimization and results

ID	Pri.	Requirement
FR-34	P0	One orchestration workflow runs validation, network analysis, allocation, routing, metric calculation and persistence in a documented order.
FR-35	P0	Show run status such as running/succeeded/partial/failed as applicable to the implementation.
FR-36	P0	Display allocation, unallocated packages and reasons, unreachable locations, route stops, road paths, cost and warnings.
FR-37	P0	Calculate metrics from real result objects. Never hard-code result metrics for appearance.
FR-38	P0	Save scenario input snapshot, algorithm configuration, output summary and detailed results.
FR-39	P1	Show run history and allow a run's details to be inspected.
FR-40	P1	Provide charts that compare actual measured values and clearly label units.

Scenario simulation

ID	Pri.	Requirement
FR-41	P0	Apply changes to a scenario copy, not directly to a historical run.
FR-42	P0	Support at least road closure, vehicle capacity reduction/unavailability, donation quantity/eligibility change and recipient unavailability.
FR-43	P0	Rerun optimization after a simulation and show a before/after comparison.
FR-44	P0	Record which changes were applied and to which scenario/run they relate.

4.6 Non-functional requirements

ID	Quality	Requirement
NFR-01	Correctness	Every saved result passes the documented integrity checks.
NFR-02	Reproducibility	A run records its input snapshot, algorithm options and random seed when relevant.
NFR-03	Explainability	Users can inspect why a package was excluded and why a location is unreachable.
NFR-04	Maintainability	Each algorithm is isolated from HTTP, React and database code.
NFR-05	Responsiveness	Typical seeded scenarios finish interactively; timings are measured, not promised in advance.
NFR-06	Robustness	Invalid input and disconnected graphs produce structured errors or partial results, not server crashes.
NFR-07	Security	Secrets remain server-side; all input is validated; SQL uses parameterized queries.
NFR-08	Accessibility	Controls have labels, keyboard focus is visible, status is not conveyed by color alone, tables have clear headings.
NFR-09	Deployment	Deployed frontend talks to the deployed API over HTTPS; the API connects securely to PostgreSQL.
NFR-10	Data honesty	Synthetic data, approximations, heuristic outputs and limitations are labeled clearly.

4.7 Product metrics

These metrics assess the implementation; no arbitrary success percentages are set before measuring.

- total eligible package value allocated;
- total package weight allocated;
- number and reasons of unallocated packages;
- vehicle capacity utilization;
- count of reachable/unreachable required stops;

- total road-network cost per route and overall;
- route cost compared with the selected baseline;
- runtime per algorithm and total workflow;
- count of validation or constraint violations (must be zero for a valid saved result);
- effect of each scenario change on allocation and route cost.

4.8 Acceptance criteria

The MVP is acceptable when all of the following are demonstrated:

1. A seeded scenario loads and can be inspected.
2. Invalid input is rejected with useful feedback.
3. A* never traverses a closed road and identifies disconnected endpoints.
4. Multiple-knapsack DP never duplicates a package and never exceeds a vehicle's capacity.
5. Ineligible and oversized packages are reported rather than silently lost.
6. Closest-pair output agrees with brute force on a test suite, within the same distance model.
7. ACO routes contain only feasible legs and displayed costs match those legs.
8. Algorithm comparisons use the same inputs and report measured values.
9. Scenario simulation does not mutate the original historical run.
10. A saved run can be reopened with its inputs and outputs intact.
11. UI values come from actual API results.
12. The project can be run locally using documented steps and deployed using documented environment variables.

4.9 Risks and mitigations

Risk	Mitigation
DP state explosion	Integer units, input bounds, validation, tests and a clear refusal for oversized inputs
ACO results vary	Record parameters/seeds, repeat trials where practical, label heuristic nature
Closest-pair coordinates are misleading	Document projection and units; distinguish geometric distance from road cost
Algorithms become tangled with UI	Keep pure algorithm modules and independent tests
Demo values are fabricated	Seeded data and result-driven UI; no mock metrics after integration
Scope becomes too large	Enforce the explicit MVP/non-goal list
Deployment database credentials leak	Backend-only environment variables and secret checks

Scope-change rule

A new feature is not automatically part of the MVP because it sounds useful. Before implementing it, describe its user value, its impact on algorithms and the data model, extra dependencies, and its effect on the CCP timeline. Update this PRD and the architecture/schema documents when a change is approved.

5 The Four Algorithms in Detail

This chapter explains each algorithm: its course category, its role, the model, pseudocode, complexity, constraints and baseline. All four are implemented by the team in plain JavaScript modules (no third-party optimization library) and are kept independent, testable and documented.

Algorithm	Input	Output	Must not be confused with
A*	Road graph, start, goal, edge costs, heuristic	One lowest-cost path (when heuristic conditions hold), or unreachable	Multi-stop ordering
Multiple-knapsack DP	Eligible packages with integerized weights/values, vehicle capacities	Feasible package-to-vehicle assignment maximizing modeled value	Route minimization
Closest pair (D&C)	Eligible points in a documented coordinate system	Closest pair and its distance under that metric	Road-network travel distance
ACO	Stops and pairwise feasible road costs	Heuristic low-cost stop ordering	Proof of global optimality

5.1 Algorithm 1 — A* Search

Course category: Greedy / search algorithm (instructor-permitted alternative to the MST role in the original brief). A* finds a low-cost path from a start location to a destination by combining the cost already travelled with a heuristic estimate of the remaining cost.

$$f(v) = g(v) + h(v)$$

- **g(v)**: cost from the start to location v.
- **h(v)**: estimated remaining cost from v to the destination.
- **f(v)**: estimated total path cost through v.

Role in NutriResQ

Calculates road paths between donors, collection hubs and recipient organizations. The interface shows the selected path, total road cost and alternative paths. A* also supplies the pairwise road costs that ACO uses.

Pseudocode

```
A_STAR(graph, start, goal, h):
    open  <- priority queue ordered by f = g + h, tie-break by stable node/road ID
    g[start] <- 0 ; push(start)
    while open not empty:
        v <- pop node with smallest f
        if v == goal: return reconstruct_path(parent, goal), g[goal]
        for each AVAILABLE edge (v, u, cost):
            if g[v] + cost < g[u]:
                g[u] <- g[v] + cost ; parent[u] <- v
                push/update(u, f = g[u] + h(u))
    return UNREACHABLE
```

Constraints and notes

- Closed or unavailable roads are excluded from the graph used for a run.
- With an admissible heuristic, A* returns an optimal path. A heuristic is admissible when it never overestimates the true remaining cost.
- For distance-based heuristics, use compatible map coordinates and edge costs. If no safe heuristic exists, use a zero heuristic (A* then behaves like Dijkstra) or qualify any optimality claim.
- Return the path, total cost, expanded-node diagnostics where useful, and an explicit unreachable result (never an empty array standing for 'no path').
- Equal-cost ties follow a documented deterministic rule such as stable location or road ID.

- **Complexity:** O(E log V) typical with a binary-heap priority queue; the number of nodes expanded depends on the heuristic quality and is never worse than Dijkstra for a consistent heuristic. Space O(V).
- **Baseline:** Dijkstra's algorithm on the same road graph.

5.2 Algorithm 2 — Multiple Knapsack using Dynamic Programming

Course category: Dynamic Programming. The algorithm distributes available food packages among several vehicles, each with its own capacity, to maximize total impact.

Let package i have integer weight w_i and impact value p_i, and let vehicle j have capacity C_j. Let $x_{ij} = 1$ if package i is assigned to vehicle j. The model is:

$$\text{maximize } \Sigma_i \Sigma_j p_i x_{ij}$$

$$\text{subject to } \Sigma_i w_i x_{ij} \le C_j \text{ for every vehicle } j \text{ and } \Sigma_j x_{ij} \le 1 \text{ for every package } i$$

The second constraint ensures each package is assigned to at most one vehicle.

DP formulation

The state is the remaining capacity of *every* vehicle at once: after considering the first i packages, $F(i; c_1,\ldots,c_k)$ is the best value achievable when vehicle j still has c_j units available.

$$F(i; c) = \max(F(i-1; c), \max \text{ over } j \text{ with } c_j \ge w_i \text{ of } [p_i + F(i-1; c - w_i \cdot e_j)])$$

Package i is either skipped, or placed on exactly one vehicle j that has room; e_j is the unit vector for vehicle j. Base case: F(0; c) = 0.

Pseudocode

```
MULTI_KNAPSACK_DP(packages, capacities C1..Ck):
    exclude ineligible/expired packages and packages heavier than every vehicle (report them)
    check state size = prod(Cj + 1) against configured limit; refuse if too large
    for i = 1..n:
        for every capacity vector c in the state space:
            best <- F[i-1][c] ; choice[i][c] <- SKIP
            for j = 1..k with c[j] >= w[i]:
                cand <- p[i] + F[i-1][c - w[i]*e_j]
                if cand > best: best <- cand ; choice[i][c] <- j
            F[i][c] <- best
    # backtrack
    c <- (C1..Ck)
    for i = n down to 1:
        if choice[i][c] != SKIP: assign package i to vehicle choice[i][c] ; c[j] -= w[i]
    return assignments, F[n][C]
```

Complexity

With n packages and integer capacities $C_1,\ldots,C_k$, a straightforward multidimensional DP needs approximately:

$$\text{Time: } O(n \cdot k \cdot \Pi_{j=1..k} (C_j + 1)) \quad \text{Space: } O(n \cdot \Pi_{j=1..k} (C_j + 1)) \text{ when the full table is kept for reconstruction}$$

Therefore choose small, sensible capacity units and a manageable number of vehicles. Avoid modelling capacities in grams if that creates huge DP tables. The state size is validated *before* allocation runs.

- **Genuine multi-knapsack required.** Do not solve an independent single-knapsack problem per vehicle: that could allocate the same package more than once. A greedy baseline may exist separately but must never replace the DP.
- Weights and capacities are integers; document the scale used to convert real weights (for example 0.1 kg per unit). No silent rounding.
- Package value is explicit and fixed for a run. Recipient urgency influences the model only through explicitly defined package values, eligibility or allocation constraints, never by silently changing a value during execution.
- Packages that fit no vehicle are excluded from the DP and reported as oversized. If total supply exceeds fleet capacity, return the best feasible partial allocation.
- Assignments are reconstructed by backtracking through the DP decisions, and reconstruction is tested separately.

- **Baseline:** greedy allocation sorted by a defined rule such as impact value per unit weight (a comparison method, not guaranteed optimal). Tiny instances are also checked against exhaustive enumeration.

5.3 Algorithm 3 — Closest Pair of Points (Divide and Conquer)

Course category: Divide and Conquer. It finds the two closest points in a set of coordinates without comparing every pair. The standard divide-and-conquer implementation runs in O(n log n) time.

Pseudocode

```
CLOSEST_PAIR(points sorted by x):
    if n <= 3: return brute_force(points)
    mid <- n / 2 ; midX <- points[mid].x
    (dL, pairL) <- CLOSEST_PAIR(left half)
    (dR, pairR) <- CLOSEST_PAIR(right half)
    d <- min(dL, dR)
    strip <- points with |x - midX| < d, sorted by y
    for each point p in strip:
        compare p with the next few points in the strip (while their y-gap < d;
        at most about 7 candidates are needed)
        update d and the best pair if a closer one is found
    return (d, best pair)
```

Role in NutriResQ

Identifies nearby donors, nearby recipient organizations, or the closest donor-recipient pair for operational analysis. The result shows the pair, their separation and their locations on the map.

- **Straight-line is not road distance.** This algorithm analyzes geographic proximity; A* determines paths through the road graph. A nearby pair is a candidate for further consideration, not automatically the best donation match.
- Define the point set, coordinate system and units first. If latitude/longitude are used, do not apply a planar Euclidean formula to degrees and label the result as kilometers; use a documented local projection for a bounded region or a justified geographic-distance model.
- Handle duplicate coordinates and fewer than two eligible points explicitly.
- **Complexity:** O(n log n) time (O(n) merge per level over log n levels), O(n) space.
- **Baseline:** brute-force checking of all pairs in $O(n^2)$; both must agree on small datasets.

5.4 Algorithm 4 — Ant Colony Optimization (ACO)

Course category: Approximation / heuristic algorithms. ACO uses simulated ants, pheromone strengths and route costs to search for good solutions to a combinatorial routing problem. In NutriResQ it determines an efficient *order* for a vehicle's collection and delivery stops using road travel costs, not straight-line distances.

Core rules

An ant at stop i chooses the next unvisited stop j with probability proportional to pheromone and desirability:

$$P_{ij} = (\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}) / \Sigma_{l \text{ allowed}} (\tau_{il}^{\alpha} \cdot \eta_{il}^{\beta}),\ \eta_{ij} = 1 / d_{ij}$$

After all ants complete a tour, pheromone evaporates and is reinforced along good tours:

$$\tau_{ij} \leftarrow (1 - \rho) \cdot \tau_{ij} + \Sigma_{ants}\, \Delta\tau_{ij},\ \Delta\tau_{ij} = Q / L_{ant} \text{ if the ant used edge } (i, j)$$

Here d_{ij} is the road cost between stops (from A*), α and β weight pheromone against desirability, ρ is the evaporation rate, Q is a constant and L_{ant} is the tour cost. Parameters and the random seed are configurable and recorded per run.

Pseudocode

```
ACO_ROUTE(stops, cost matrix d from A*, params: ants, iterations, alpha, beta, rho, Q, seed):
    initialize pheromone tau on all feasible pairs
    best <- none
    repeat for each iteration:
        for each ant:
            build a tour: start -> repeatedly choose next unvisited stop by probability P_ij
            (never use a pair with no road path; if a stop cannot be connected, mark it unreachable)
            compute tour cost from road legs
        evaporate: tau <- (1 - rho) * tau
        deposit: for each ant, add Q / cost along its tour
        update best if a cheaper feasible tour appeared
    return best tour, cost, convergence history
```

- Respect the locations assigned to each vehicle, road availability and the defined pickup/delivery rules. If food expiry is part of the model include time feasibility; otherwise filter ineligible or expired batches before optimization.
- **Example:** ACO might choose the order Donor A → Donor C → Recipient B. A* then calculates the road path between each consecutive pair, and these costs evaluate the overall tour.
- ACO is a **heuristic**: it may find a good route without proving global optimality. Results vary between runs, so repeat trials and report best, average and worst.
- **Complexity (typical):** O(iterations · ants · n^2) for n stops, plus building the cost matrix with A*. Input, iteration and runtime limits protect the server.
- **Baseline:** nearest-neighbor routing on the same locations, road costs and constraints.

5.5 Why use both A* and ACO?

They solve different routing problems. **A*** finds a path through a road network between a specific origin and destination. **ACO** searches for a good order in which to visit multiple stops. The orchestrator uses A* costs for every required stop pair, feeds that matrix to ACO, and then validates the final route cost by recalculating it from the road legs.

Algorithm	Time complexity	Space	Baseline and its complexity
A*	O(E log V) typical	O(V)	Dijkstra, O(E log V)
Multiple-knapsack DP	O(n·k·Π(C_j+1))	O(n·Π(C_j+1))	Greedy by value/weight, O(n log n)
Closest pair (D&C)	O(n log n)	O(n)	Brute force, O(n^2)
ACO	O(iterations·ants·n^2) typical	O(n^2)	Nearest neighbor, O(n^2)

Note: Complexity figures for A* and ACO are typical expected bounds. The final report must derive and state the bounds for the exact implementations, and runtimes must be measured, not assumed.

6 Data, Constraints and Modeling Rules

6.1 Suggested CCP dataset

The initial dataset must be large enough to demonstrate the algorithms and satisfy the scale required by the course brief. These are proposed starting values, not field data.

15	10	5	25+
Donor locations	Recipient organizations	Delivery vehicles	Road connections

Together these give 25 distinct donor and recipient locations before adding optional hubs. The minimum required by the CCP is at least 10 locations and 20 roads for the integrated worked example.

6.2 Entities and attributes

Entity	Important attributes
Location	ID, name, type, latitude, longitude, urgency or priority
Road	Start location, end location, travel cost, availability
Donor	Location, donation details, operating status
Recipient	Location, food demand, priority, storage or handling limits
Food batch	Weight, impact value, quantity, eligibility and expiry
Vehicle	Capacity, availability, assigned packages and route
Scenario	Input configuration, road changes, algorithm settings and results

6.3 Core constraints to implement

Road network

- A road marked unavailable must not be used by A* or ACO.
- Detect unreachable locations and report them by name.
- Use a documented, deterministic tie-breaking rule when paths have equal cost (for example stable location or road ID).
- When three previously available roads become unavailable, update affected paths and routes. Compare an incremental repair strategy with full recomputation; recompute from scratch only as a fallback and document the limits of the incremental method.

Food allocation

- Each package may be assigned to at most one vehicle; each vehicle stays within capacity.
- Package weights and impact values are fixed inputs for a given run.
- Packages that cannot fit into any vehicle are excluded from the DP and reported.
- If total demand exceeds fleet capacity, return the best feasible partial allocation rather than failing.
- Reconstruct and display actual package-to-vehicle assignments, not just the maximum value.

Delivery routing

- Use the road graph and available connections; route only the stops assigned to the relevant vehicle.
- Track travel cost and vehicle capacity.
- If expiry windows are enforced, verify the proposed deliveries can meet them.
- Handle unreachable stops rather than silently treating them as reachable.

Geographic analysis

- Use valid coordinates and a consistent distance calculation.
- Distinguish straight-line proximity from actual road travel.
- Test the closest-pair algorithm against brute force on small datasets.

6.4 Seed data must demonstrate

The synthetic seed dataset should include enough variety to demonstrate: at least one route with multiple stops; a closed road; a disconnected or unreachable case; a donation that expires or is ineligible; a package too large for the available fleet; more eligible food than total fleet capacity; and multiple vehicles with different capacities.

7 The Three Main Interactive Features

These features make NutriResQ feel like a genuine optimization product rather than a basic university CRUD project. Every value displayed must come from real algorithm and API results, never from invented numbers.

7.1 Interactive Optimization Dashboard

The main operations screen. It summarizes the current scenario and lets the user run the optimization workflow. The four headline indicators are calculated from the current run: **food allocated**, **total route cost** (distance or travel-cost units), **fleet utilization** (load versus total capacity) and **optimization runtime** (measured execution time). Before a run, these show a dash rather than invented values.

Dashboard components

Component	What it shows
Interactive map	Donors, recipient organizations, hubs, available roads, closed roads and calculated routes. Users can select a location, inspect its details and see route overlays change after a new run.
Run optimizer action	Executes the selected algorithm pipeline on the current scenario.
Allocation panel	Which packages go on which vehicles, unused capacity and unallocated packages with reasons.
Route panel	Stop order, route cost and unreachable destinations.
Network panel	Connectivity information and the paths selected by A*.
Analytics charts	Vehicle utilization, package allocation, route costs and algorithm runtimes.
Run history	Open a previous scenario or run and inspect its results.

7.2 Algorithm Comparison Laboratory

The academic and technical heart of NutriResQ. It provides evidence that the algorithms do useful work by comparing each against a simpler baseline on identical inputs. A fair comparison uses the same inputs and constraints; a faster algorithm is not called better if it produces an invalid or infeasible solution.

Experiment	Comparison	Metrics reported
1. A* pathfinding	A* vs Dijkstra on the same road graphs	Path cost and validity; execution time and nodes expanded; behaviour when roads close or destinations become unreachable
2. Multiple-knapsack DP	DP vs greedy allocation (value per unit weight)	Total impact value; total weight allocated; vehicle capacity utilization; runtime and feasibility
3. Closest pair	Divide and conquer vs brute force	Closest distance and pair; execution time as locations increase; agreement on small datasets
4. ACO delivery routing	ACO vs nearest neighbor	Total route cost; best, average and worst across repeated runs; runtime and convergence over iterations

ACO is stochastic, so trials are repeated and the actual measured results are reported. **Why it matters:** it connects the working application to DAA concepts: correctness, complexity, trade-offs and experimental evaluation.

7.3 Scenario Simulator

Lets a user change the conditions of the network and see the effects on allocation, connectivity and delivery routes. Settings modify a *copy* of the active scenario; running the simulation calculates real results, and the original scenario is preserved so before and after can be compared. Example controls: additional road closures (0 to 5), vehicles unavailable (0 to 2) and donation supply reduction (0% to 80%).

Recommended test scenarios

#	Scenario	Action and expected check
1	Road failure	Close three road connections, update affected paths and compare incremental repair with full recomputation.
2	Reduced fleet	Remove a vehicle and determine which packages can still be carried.
3	Food shortage	Reduce available donations and calculate the best feasible partial allocation.
4	Oversized package	Add a package exceeding every vehicle's capacity and verify it is excluded correctly.
5	Disconnected network	Isolate a recipient and report that it cannot be reached under the current road configuration.
6	Urgency sensitivity	Remove the two highest-urgency recipient locations, rerun the relevant algorithms and compare results.
7	Expiry scenario	Mark a donation expired or ineligible and verify it is excluded before allocation and routing.

Scenarios 6 and 7 are especially relevant to the assignment's sensitivity-analysis requirement.

8 System Workflow and Optimization Pipeline

The system needs a coherent execution pipeline so the four algorithms act as one decision-support workflow. The high-level flow is shown below, followed by the detailed orchestration order.

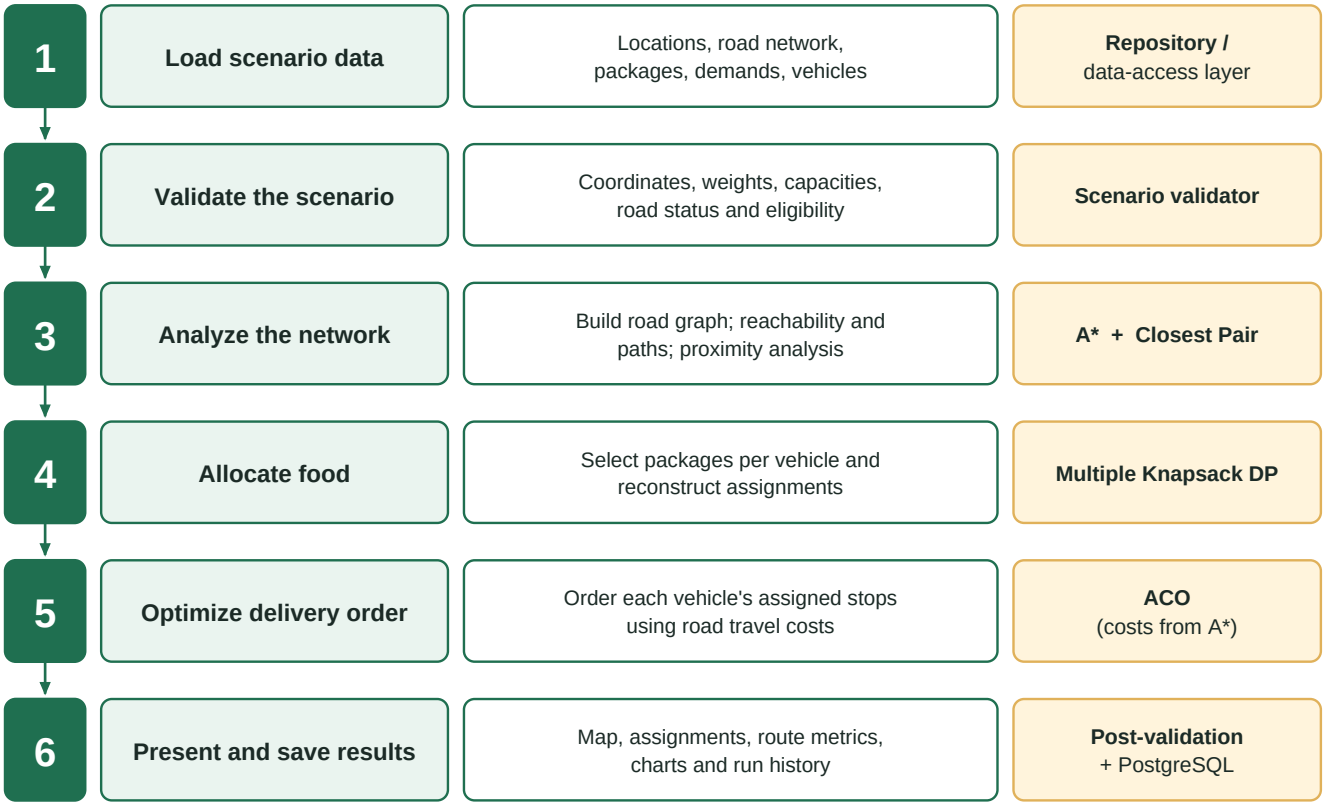


Figure 8.1 — The six-stage NutriResQ pipeline and the component responsible for each stage

8.1 Important rules of the pipeline

- The allocation output determines which packages each vehicle carries and which recipients need to be served.
- The routing stage uses the resulting stops and road costs; it must not visit arbitrary locations that were not assigned to the vehicle.
- If a recipient is unreachable, the system flags the infeasibility and shows which allocations or routes are affected.
- Closest Pair provides proximity analysis; it does not independently determine the best recipient assignment.

- The final report records scenario inputs, algorithm settings, results and measured execution times so experiments are reproducible.

8.2 Detailed orchestration sequence

#	Step	What happens
1	Load immutable input snapshot	Resolve scenario data and record its version/snapshot.
2	Validate	Check references, values, coordinate fields, edge weights, capacities, dates and algorithm limits. Stop on fatal errors; retain non-fatal warnings.
3	Build road graph	Exclude closed roads and apply the documented directionality/cost model.
4	Network analysis	Run A* for required origin/destination pairs; record unreachable pairs explicitly; run closest-pair analysis on the eligible location set.
5	Donation eligibility	Exclude expired or ineligible packages; identify packages too large for every available vehicle.
6	Allocation	Run multiple-knapsack DP and reconstruct assignments; validate unique assignment and capacity constraints.
7	Stop construction	Derive route stops from the allocation and documented pickup/delivery semantics; never route unassigned or ineligible work.
8	Travel-cost matrix	Use A* road costs for every required stop pair. An unreachable pair is not a normal numeric cost and must be handled explicitly.
9	Route ordering	Run ACO on feasible costs. If a required stop cannot be connected, mark the route partial/infeasible and explain why.
10	Post-validation	Recalculate costs from road paths and verify all constraints.
11	Metrics and persistence	Calculate actual metrics, save run metadata and detailed results transactionally, return the saved run identifier.
12	Presentation	Frontend renders returned results, warnings and metrics.

Note: Allocation and routing are related but distinct. The allocation objective does not automatically minimize travel cost, and the UI must make that trade-off visible.

8.3 End-to-end activity flow

Stage	Activities and decisions
Start	Coordinator opens NutriResQ; the dashboard and saved scenarios load; the coordinator selects an existing scenario or creates a new one; scenario data (locations, roads, food batches, recipient requirements, vehicle capacities) loads; inputs may be edited.
Validation	Check IDs, coordinates, road costs/status, food weights/values, vehicle capacities and food eligibility. **Decision: valid?** NO: show errors, correct inputs and revalidate. YES: prepare a working copy, record run configuration and start timing.
Stage 1: network	Build the road graph from available roads; run A*; check reachability. **Decision: reachable?** NO: record the unreachable destination and affected operations. YES: store path and cost. Then run Closest Pair and store the closest locations and straight-line distance.
Stage 2: allocation	Filter expired/ineligible batches; identify and exclude oversized packages; run multiple-knapsack DP; backtrack to reconstruct assignments; verify no duplicates, capacity respected and feasible partial result. **Decision: feasible?** NO: flag inconsistency. YES: store assignments, impact and capacity.
Stage 3: routing	Derive pickup and delivery stops; build road-cost information per leg; run ACO; evaluate candidate routes using road costs; validate feasibility and unreachable stops; record stop order, cost and status.
Stage 4: results	Calculate final metrics (food allocated, total impact, route cost, fleet utilization, runtime, unreachable locations); save the run, configuration snapshot, assignments and routes; display dashboard, map, allocations, routes and analytics.
Next action	Coordinator chooses: view a saved run; run algorithm comparisons (A* vs Dijkstra, DP vs greedy, Closest Pair vs brute force, ACO vs nearest neighbor); or configure a simulation (modify a working copy, validate, rerun affected algorithms, save and compare). **Continue working?** YES: return to dashboard. NO: end.

8.4 Feasibility and error model

The system distinguishes the following outcomes. They must never be collapsed into an empty array or a generic HTTP 500.

Outcome	Meaning
Validation failure	The request or dataset is malformed.
Unreachable	No path exists through available roads for a required pair.
Ineligible	A donation cannot be considered under the configured rules.
Oversized	A package cannot fit any available vehicle under the modeled capacity.
Partial	A feasible result exists, but not all desired work can be completed.
Algorithm failure	An implementation error or configured limit prevented a valid result.
Success	Outputs pass the documented post-validation checks.

9 System Architecture

9.1 Architectural goals

Keep algorithmic correctness independent of the interface; allow results to be tested without a browser or database; persist reproducible runs; and allow a UI component to be replaced without rewriting the optimization engine. The MVP is a **modular monolith**: one frontend application, one Express API, one PostgreSQL database and independent algorithm modules inside the backend codebase. No microservices without a demonstrated need.

9.2 Component and deployment view

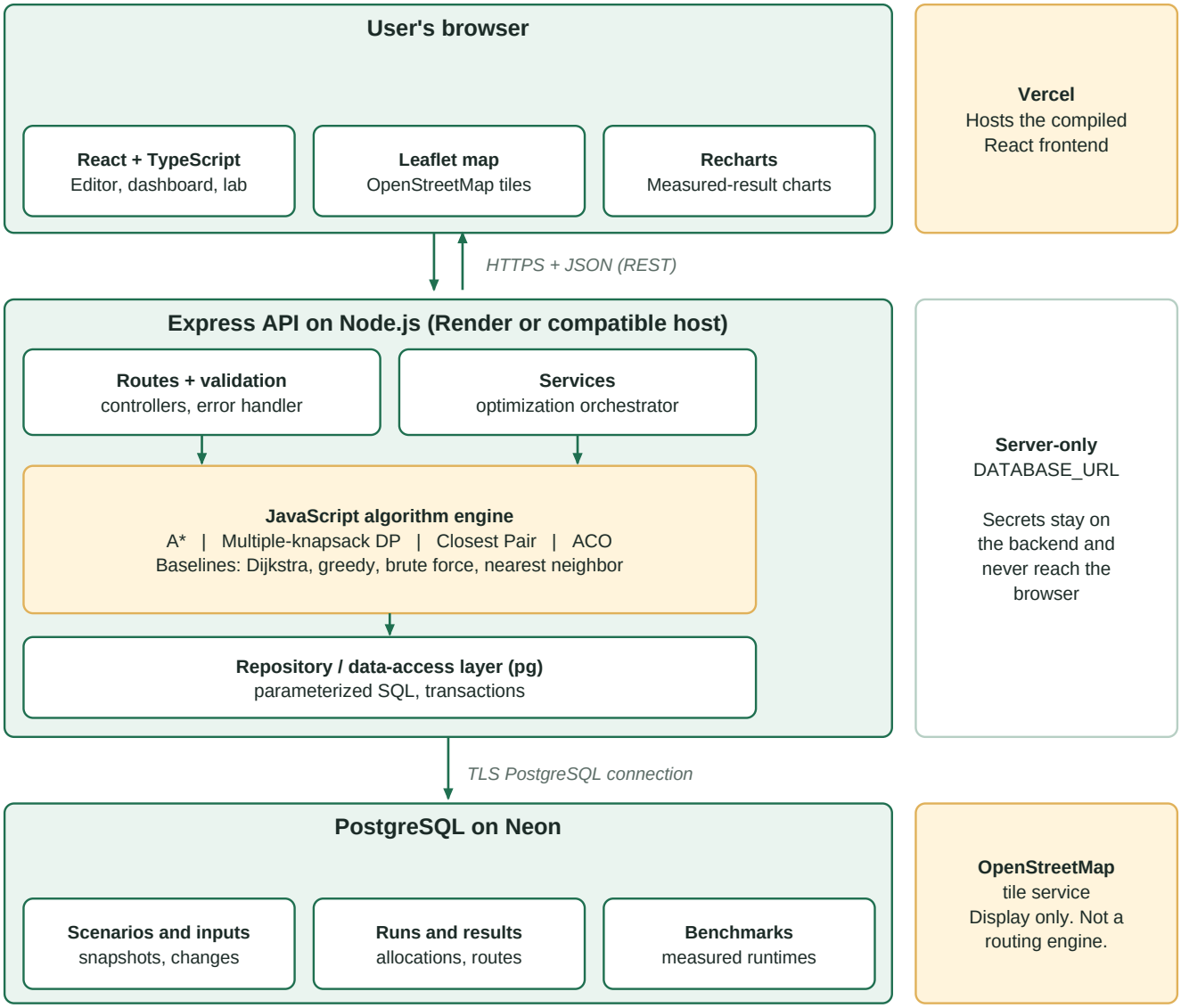


Figure 9.1 — Browser, Express API with algorithm engine, and PostgreSQL on Neon, plus supporting services

9.3 Deployment topology

- **Frontend:** React application deployed to Vercel.
- **Backend:** Express API on Render or another compatible Node.js host.
- **Database:** PostgreSQL hosted on Neon.
- **Map tiles:** Leaflet requests tiles from an OpenStreetMap-compatible provider in the browser, subject to that provider's usage policy.
- **Communication:** the frontend calls the API over HTTPS using a configured base URL; the backend connects to PostgreSQL with a server-only connection string.
- Do not put the Neon connection string in frontend environment variables. A variable exposed to browser code is not secret merely because its name begins with VITE_.

9.4 Responsibilities

Layer	Responsibilities
Frontend	Collects and edits scenario inputs; calls API endpoints; renders map, allocations, routes, metrics, warnings and history; does client-side convenience validation but never replaces backend validation; never runs authoritative optimization or connects to PostgreSQL.
Express API	Validates and normalizes requests; applies business rules; calls services and algorithms; returns stable JSON shapes and suitable HTTP status codes; handles errors centrally; never trusts client-submitted metrics.
Optimization service	Coordinates the sequence of steps and combines outputs. Contains no UI logic or SQL; inputs and outputs are plain data objects that can be unit tested.
Algorithm modules	Operate on explicit inputs and return explicit outputs; do not read environment variables, query the database or send HTTP responses; do not mutate shared scenario data; expose diagnostics for checks and benchmarks; are tested independently.
Repository layer	Performs parameterized SQL; maps rows to application objects; persists snapshots, run metadata, allocations, routes and benchmarks; handles transaction boundaries.
PostgreSQL	Stores scenarios, source records, changes, runs, detailed outputs and benchmark measurements; migrations are versioned and committed.

9.5 Scenario and run immutability

- Editing a scenario affects future runs, not prior snapshots.
- Simulation starts from a copy and records each change.
- A run refers to the exact inputs used, not only a mutable scenario ID.
- Algorithm version/configuration and random seed are recorded when relevant.
- Reopening a run never silently recomputes it with newer algorithm code.

9.6 Performance and observability

Start with a synchronous workflow for small seeded scenarios if measured execution time is acceptable. Set input limits for DP and ACO so a request cannot monopolize the server. If later measurements show runs are too long for request/response, move execution to a background job pattern without changing the pure algorithm interfaces. Do not add a queue or distributed infrastructure before it is needed.

Record safe structured logs: request/run identifier, scenario identifier and snapshot, algorithm name and configuration, duration and status, validation or feasibility warnings, and benchmark trial/seed. Never log database passwords, connection strings, session tokens or unnecessary personal information.

9.7 Architecture decisions to record before implementation

- frontend build tool and package versions;
- API route conventions and validation library;
- test framework;
- coordinate projection and distance units for closest pair;
- road edge directionality and edge-cost units;
- route semantics for pickup and delivery stops;
- integer scaling and limits for multiple-knapsack DP;
- ACO parameter defaults and random seed policy;
- authentication scope for the first release; and
- how a run snapshot is represented and versioned.

10 Technology Stack

The approved stack is the authoritative allowlist for the project. It prevents silently replacing the database, changing the algorithm language, adding unnecessary frameworks, or moving secret-bearing logic into the browser.

Layer	Approved choice	Responsibility
UI framework	React	Component-based interactive interface
Frontend language	TypeScript	Typed UI, API models and component props
Styling	Tailwind CSS	Utility-based styling and responsive layouts
Interactive maps	Leaflet	Render locations, roads, paths and route stops
Map tiles	OpenStreetMap-compatible tiles	Geographic basemap display
Charts	Recharts	Visualize measured metrics and comparisons
Server runtime	Node.js (supported LTS)	Run the API and algorithm modules
API framework	Express	HTTP routes and middleware
Algorithms	Plain JavaScript modules	A*, multiple-knapsack DP, closest pair, ACO and baselines
Database	PostgreSQL on Neon	Persistent application data
DB driver	pg	Server-side connections and parameterized queries
Frontend hosting	Vercel	Deploy the frontend
API hosting	Render or compatible Node host	Deploy the Express API
Version control	Git + GitHub	Source history, review and collaboration

10.1 Additional recommended tooling

Purpose	Recommendation
Frontend build tooling	Vite
API communication	Fetch API
Input validation	Zod or another schema-validation library
Backend tests	Node.js built-in test runner or Vitest
Frontend tests	Vitest + React Testing Library
API documentation	OpenAPI/Swagger (optional)
Code quality	ESLint + Prettier
Environment variables	.env locally; host-managed variables in deployment

Redux, Redis, microservices, AI integration or a separate Python service are not needed for the initial version; they would add complexity without being required for the DAA goals. Concrete package versions are chosen at repository initialization and the lockfile is committed.

10.2 Non-negotiable constraints

1. **PostgreSQL stays PostgreSQL.** No replacement by Supabase, Firebase, MongoDB or SQLite without explicit approval.
2. **Algorithms use JavaScript** in Node.js, including the baselines.
3. **No database access from the browser.** Only the backend uses pg and the Neon connection string.
4. **No fake optimization output.** UI metrics and route paths come from real algorithm/API results.
5. **Map display is not route computation.** Leaflet and OSM tiles show context; A* and ACO are implemented by the team.
6. **No microservices** for the MVP; use a modular monolith.

7. **No unapproved dependency sprawl.** Explain use case, alternatives, security and bundle impact before adding a package.
8. **No frontend secrets.** Database credentials and private keys never enter client bundles.
9. **Same algorithm implementation everywhere.** Standalone endpoints and the integrated workflow call the same tested modules.
10. **No silent scope changes.** Follow the PRD.

10.3 Database, frontend and backend rules

- Parameterized SQL for all user-influenced values; versioned SQL migrations; a transaction when saving a run with its allocations and routes; secure connection settings; never commit .env files or credentials.
- Frontend: TypeScript types for API responses; a small shared API client rather than scattered fetch calls; backend results are authoritative; algorithm logic stays out of React components; explicit loading, empty, error, partial and success states; map components isolated so rendering problems do not break other views; avoid gradients unless approved.
- Backend: routes/controllers for HTTP, services for orchestration, algorithm modules for computation, repositories for database access; validate every external request; centralized error handling with consistent shapes; never accept client-supplied totals or route costs as trusted; no stack traces or database errors to clients; secrets in a server-only config module; seed recorded where randomness is intrinsic.

10.4 Environment variables

Scope	Variable	Notes
Backend	NODE_ENV, PORT	Runtime mode and port binding
Backend	DATABASE_URL	Backend-only; never exposed to the browser
Backend	FRONTEND_ORIGIN (or CORS allowlist)	CORS restricted to the expected frontend origin
Frontend	VITE_API_BASE_URL (if using Vite)	Only values safe to expose may use public prefixes

.env.example contains placeholders only. Required variables are validated at startup, failing with a clear, secret-free message. Local .env files are ignored by Git.

10.5 Deployment

Frontend: deploy to Vercel with its public API base URL pointing to the deployed backend; the production build contains no credentials. **Backend:** deploy Express to Render or a compatible host with secrets set in the host's configuration, CORS limited to the frontend origin, and a health endpoint that exposes no sensitive details; confirm port binding, start command and Node version. **Database:** Neon PostgreSQL with migrations applied in order, least-privilege credentials where supported, and connection limits/pooling confirmed. Provider details and free-tier limits change, so verify them at implementation time.

10.6 Not approved by default

- Supabase or Firebase; MongoDB; Python-based algorithm services; a microservice per algorithm.
- A third-party routing API as a substitute for implementing A* and ACO.
- A UI component framework added only for convenience.
- Authentication providers or role systems before the MVP authentication decision.
- Queues, Redis or background workers before measured need; AI/ML features unrelated to the four algorithms.

10.7 Technology decision log

Decision	Status
React + TypeScript, Tailwind CSS, Leaflet + OSM tiles, Recharts	Approved
Node.js + Express, JavaScript algorithm modules	Approved
Neon PostgreSQL + pg	Approved
Vercel frontend; Render or compatible Node host	Approved targets (provider to confirm)
Concrete package versions	Pending repository initialization
Frontend build tool	Confirm before setup
Test framework	Confirm before algorithm implementation
Authentication for MVP	Not required by current scope; decide if access control is needed

11 Database Design

PostgreSQL on Neon is the system of record. It stores both the real-world entities and the reproducible experiments. Donors, recipients and vehicles are data records, not login roles.

11.1 Design principles

- Normalized source records with explicit foreign keys.
- A run preserves a snapshot of the exact inputs used.
- Detailed results are stored separately from mutable scenario inputs.
- CHECK and NOT NULL constraints for basic integrity; application validation for cross-record and algorithm rules.
- Versioned migrations; no manual changes to the hosted schema without recording them.
- Parameterized SQL through pg; timestamptz values stored consistently in UTC; no credentials or secrets in domain tables.

11.2 Entity relationship overview

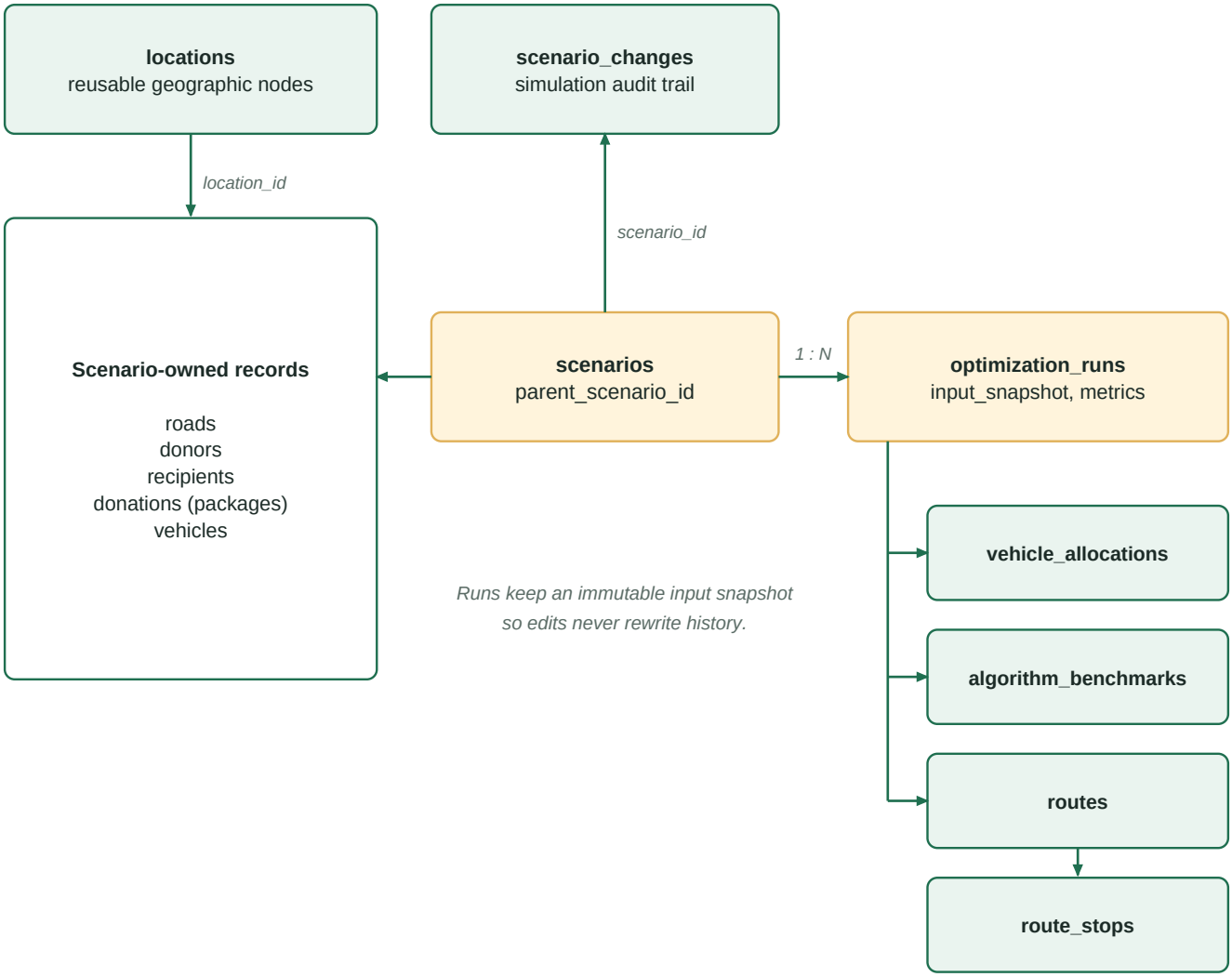


Figure 11.1 — Logical entity relationships (scenario-owned records model)

Modeling choice: a location is a reusable geographic record, while a scenario controls which road, donor, recipient, donation and vehicle states are active in an experiment. The MVP proposal uses scenario-owned operational records (simplifying snapshots) with locations as reusable records. Pick one model before writing migrations and keep it consistent.

11.3 Table summary

Table	What it stores
users	Optional accounts and roles, only if authentication is implemented
locations	Coordinates, names, types and priority attributes
roads	Connected locations, travel cost, availability and road status
donors	Food-donation sources and their locations
recipients	Organization details, demand, priority and availability
donations	Donor, package details, weight, value, quantity and expiry
vehicles	Capacity, availability, start location and metadata
scenarios	Saved configurations and scenario names
scenario_changes	Road closures, supply changes and other simulation modifications
optimization_runs	Algorithm settings, status, duration, snapshot and summary metrics
vehicle_allocations	Package-to-vehicle assignments for each run
routes	One vehicle route per run with status and cost
route_stops	Stop sequence, location and calculated leg costs
algorithm_benchmarks	Baseline results, runtimes and comparison metrics

11.4 Table definitions

The definitions below are a logical schema proposal, not a migration file. Final SQL types, names and constraints are verified when migrations are written. Use one key strategy (UUID) consistently.

locations

Geographic nodes such as donor sites, recipient sites or other network points.

Column	Type	Rules / meaning
id	uuid	Primary key
name	text	Required, non-empty
location_type	text	donor, recipient, hub, other
latitude	double precision	Required; -90 to 90
longitude	double precision	Required; -180 to 180
projected_x / projected_y	double precision	Optional projected coordinates for planar closest pair; use only if projection and units are documented
created_at / updated_at	timestamptz	Required defaults

- Never compute kilometer distances directly from latitude/longitude degree differences.

roads

A graph edge between locations.

Column	Type	Rules / meaning
id	uuid	Primary key
scenario_id	uuid	FK to scenarios if roads are scenario-owned
from_location_id / to_location_id	uuid	FKs to locations
cost	numeric / double	Required, finite, non-negative
cost_unit	text	For example km or cost_points; keep consistent

Column	Type	Rules / meaning
is_available	boolean	False means closed/unavailable
is_bidirectional	boolean	Defines how the graph builder interprets the edge
created_at / updated_at	timestamptz	Required

- from_location_id must differ from to_location_id unless self-loops are deliberately supported; prevent duplicate edges per the chosen directionality policy.
- Closed roads stay stored but are excluded from the graph for that run; edge cost units must match those used by A* and ACO.

donors

A food-donation source.

Column	Type	Rules / meaning
id, scenario_id	uuid	Primary key; FK to scenarios
location_id	uuid	FK to locations
name	text	Required
donor_type	text	restaurant, bakery, supermarket, other
is_active	boolean	Required
created_at / updated_at	timestamptz	Required

- No login credentials are stored in this table.

recipients

A recipient organization and its modeled need and priority.

Column	Type	Rules / meaning
id, scenario_id	uuid	Primary key; FK to scenarios
location_id	uuid	FK to locations
name	text	Required
recipient_type	text	shelter, food_bank, community_kitchen, welfare_org, other
demand_units	integer / numeric	Non-negative; units documented
priority_score	integer / numeric	Documented range and meaning
is_available	boolean	Required
created_at / updated_at	timestamptz	Required

- If package value depends on recipient priority, document the formula and make sure priority is not applied twice.

donations

A donation batch or package considered by allocation. Rows are indivisible packages unless split quantities are explicitly modeled.

Column	Type	Rules / meaning
id, scenario_id	uuid	Primary key; FK to scenarios
donor_id	uuid	FK to donors
package_name	text	Required
weight_units	integer	Positive integer in the documented DP scale
value_score	integer	Non-negative integer objective value for DP
recipient_id	uuid, nullable	Optional intended recipient for directed donations
available_at / expires_at	timestamptz	Optional; define whether equality with run time means expired

Column	Type	Rules / meaning
is_eligible	boolean	Required
status	text	available, expired, reserved, unavailable
created_at / updated_at	timestamptz	Required

- weight_units > 0 and value_score >= 0; expiry is evaluated against a documented run timestamp.
- A package that fits no available vehicle is reported as oversized. Fractional weights need an approved conversion scale (for example 0.1 kg per unit), never implicit rounding.

vehicles

A vehicle available for allocation and routing.

Column	Type	Rules / meaning
id, scenario_id	uuid	Primary key; FK to scenarios
name	text	Required
capacity_units	integer	Positive; same weight scale as DP
start_location_id	uuid	FK to locations
is_available	boolean	Required
created_at / updated_at	timestamptz	Required

- The model has one capacity dimension (weight). Adding volume or refrigeration means revising the algorithm model and schema together.

scenarios

A named experiment or data configuration.

Column	Type	Rules / meaning
id	uuid	Primary key
name, description	text	Name required; description optional
status	text	draft, ready, archived
parent_scenario_id	uuid, nullable	Self-FK for a simulated or copied scenario
created_at / updated_at	timestamptz	Required

scenario_changes

Records changes applied during simulation.

Column	Type	Rules / meaning
id, scenario_id	uuid	Primary key; FK to the simulated scenario
source_run_id	uuid, nullable	FK to the run used as comparison baseline
change_type	text	close_road, reduce_capacity, expire_donation, disable_vehicle, disable_recipient
target_entity_type / target_entity_id	text / uuid	Entity changed; validate it belongs to the scenario
before_value / after_value	jsonb	Audit of prior and new values; not a substitute for normalized data
created_at	timestamptz	Required

optimization_runs

One record per integrated optimization attempt.

Column	Type	Rules / meaning
id, scenario_id	uuid	Primary key; FK to scenarios
parent_run_id	uuid, nullable	Self-FK for before/after lineage
status	text	running, succeeded, partial, failed
input_snapshot	jsonb	Immutable normalized inputs used for the run
algorithm_config	jsonb	Algorithm names, parameters, versions and seed
metrics	jsonb	Summary metrics with documented units
warnings	jsonb	Structured warning array
error_code / error_message	text, nullable	Safe stable code and summary; no secrets or stack traces
started_at / completed_at / created_at	timestamptz	Completion set on finish or failure

- input_snapshot is central to reproducibility: reopening a run must not rebuild inputs from mutable current records alone.

vehicle_allocations

One row per package assignment made in a run.

Column	Type	Rules / meaning
id, run_id	uuid	Primary key; FK to optimization_runs
vehicle_id / donation_id	uuid	FKs to vehicle and donation records (or snapshot references)
assigned_weight_units	integer	Positive; equals package weight for indivisible packages
value_score	integer	Value used by the optimizer
created_at	timestamptz	Required

- Unique (run_id, donation_id) for indivisible packages; no row for unassigned packages.
- Post-validation confirms per-vehicle weight totals stay within snapshot capacity. Keep names and key facts in the snapshot so edited or deleted records do not break old runs.

routes

One row per vehicle route in a run.

Column	Type	Rules / meaning
id, run_id	uuid	Primary key; FK to optimization_runs
vehicle_id	uuid	FK to vehicle record or snapshot reference
status	text	complete, partial, infeasible, not_required
total_cost / cost_unit	numeric / text	Non-negative for feasible legs; same unit as the graph
algorithm_name / algorithm_config	text / jsonb	For example aco, with parameters and seed
created_at	timestamptz	Required

- Route cost is always recalculated from road legs; a client-supplied total is never trusted.

route_stops

Ordered stops for a route.

Column	Type	Rules / meaning
id, route_id	uuid	Primary key; FK to routes
sequence_number	integer	Positive; unique per route
location_id	uuid	FK to locations or snapshot location

Column	Type	Rules / meaning
stop_type	text	start, pickup, delivery, other
related_donation_id / related_recipient_id	uuid, nullable	Link to the donation or recipient served
leg_cost_from_previous	numeric, nullable	Null for the first stop; documented units
leg_path	jsonb	Ordered road node/edge identifiers
created_at	timestamptz	Required

- Unique (route_id, sequence_number). Define pickup/delivery semantics first; if only delivery stops are modeled, document that simplification.

algorithm_benchmarks

Comparison measurements.

Column	Type	Rules / meaning
id, run_id, scenario_id	uuid	run_id nullable; FK to related run and scenario
algorithm_name / baseline_name	text	Baseline nullable
input_fingerprint	text	Identifies comparable normalized inputs
configuration	jsonb	Parameters, seed, repeat/trial details
result_summary	jsonb	Objective value, path cost, pair distance or relevant result
runtime_ms	numeric	Non-negative measured runtime
memory_bytes	bigint, nullable	Only if measured reliably
trial_number	integer	Positive if repeated trials are stored
created_at	timestamptz	Required

- Algorithm and baseline must run on equivalent inputs; runtime alone never proves better solution quality.

11.5 Relationships

- One scenario has many runs and scenario changes; it contains or references locations, roads, donors, donations, recipients and vehicles per the chosen model.
- One run belongs to one scenario and may have a parent run for comparison lineage.
- One run has many allocations, routes and benchmark rows; a route has ordered stops.
- Each donation is assigned at most once per run in the indivisible-package model.

11.6 Units and constraints to decide before migrations

Topic	Decision needed
Road cost	Distance, time or abstract cost points; all edges and route totals must be compatible.
Package weight	Unit and integer scaling for DP (for example 3.4 kg becomes 34 units at 0.1 kg per unit).
Package value	A non-negative integer score or another precisely defined objective.
Recipient demand	Units and how demand affects selection or fulfillment metrics.
Coordinates	Latitude/longitude for display; projected coordinates or a documented geographic method for closest pair.
Expiry semantics	Whether expires_at <= run start means expired.
Directionality	Whether each road row is directed or bidirectional and how reverse edges are built.
Indivisibility	If packages can be split, change constraints, assignment rows and the DP model together.

11.7 Index recommendations

Add indexes based on actual query patterns and confirm with measurements. Likely candidates: roads(scenario_id, is_available); roads(from_location_id) and roads(to_location_id); donations(scenario_id, status, expires_at); vehicles(scenario_id, is_available); recipients(scenario_id, is_available); optimization_runs(scenario_id, created_at DESC); vehicle_allocations(run_id, vehicle_id); routes(run_id, vehicle_id); route_stops(route_id, sequence_number); algorithm_benchmarks(scenario_id, created_at DESC). Avoid indexes without a use case, since they increase write and storage cost.

11.8 Transactions, migrations and seed data

Saving a completed run uses one transaction: insert or update run status and snapshot; insert allocations; insert routes and ordered stops; insert benchmark rows if part of the operation; update final metrics, status and completion time; commit only if integrity checks pass. On failure roll back related writes and persist a safe failure status separately if needed. A run is never left marked succeeded with missing result rows.

Migration order: (1) locations; (2) scenarios; (3) scenario-owned tables (roads, donors, recipients, donations, vehicles); (4) scenario_changes; (5) optimization_runs; (6) vehicle_allocations, routes, route_stops; (7) algorithm_benchmarks; (8) indexes, check constraints and deferred foreign keys. Seed only clearly labeled synthetic data.

11.9 Privacy and access

The MVP operates without donor or recipient accounts. Do not create authentication tables merely because those entities exist. If authentication is added later, design it separately (password/session handling, roles, access rules) and never store plaintext passwords or session secrets in domain tables. The API must prevent a caller from editing records by submitting another scenario's IDs; enforce scenario ownership in services and database constraints.

12 API Design

All endpoints are prefixed with `/api` and use JSON. The main requirement is that the API invokes actual algorithm implementations and returns structured results, never hard-coded sample output. Standalone algorithm endpoints share validators and modules with the integrated workflow.

Method and path	Purpose
GET /api/health	Lightweight health check; exposes no secrets
GET /api/scenarios	List scenarios
POST /api/scenarios	Create a scenario
GET /api/scenarios/:id	Retrieve a scenario and relevant input data
PATCH /api/scenarios/:id	Update supported scenario data
GET /api/locations?scenarioId=	Retrieve scenario locations
GET /api/roads?scenarioId=	Retrieve scenario road edges
POST /api/optimize	Run the integrated optimization for a scenario or explicit snapshot
GET /api/runs/:id	Retrieve run metadata and results
GET /api/scenarios/:id/runs	List runs for a scenario
POST /api/algorithms/astar	Run A* in isolation for development/evaluation
POST /api/algorithms/knapsack	Run multiple-knapsack DP in isolation
POST /api/algorithms/closest-pair	Run closest-pair analysis in isolation
POST /api/algorithms/aco	Run ACO in isolation
POST /api/benchmarks	Compare an algorithm and its baseline on the same inputs
POST /api/scenarios/:id/simulate	Create a changed scenario copy and rerun or prepare a simulation

12.1 Response principles

- Consistent error responses with a stable error code, readable message and field details where relevant.
- Distinguish invalid input from unreachable or infeasible outcomes.
- Return units with numeric metrics.
- Never return stack traces, credentials or raw SQL errors.
- Paginate history if it outgrows the demo dataset.
- Use request or run IDs to help diagnose failures.
- Debug-only endpoints are not exposed in production without a deliberate reason.

Illustrative response shape (a placeholder, not a fixed contract; real responses use computed values):

```
{
  "runId": "example-run-id",
  "status": "succeeded",
  "metrics": {
    "allocatedPackageValue": <computed>,
    "totalRoadCost": <computed>,
    "unreachableStopCount": <computed>
  },
  "warnings": [],
  "resultsUrl": "/api/runs/example-run-id"
}
```

13 Use Case Model

The only required interactive actor is the Operations Coordinator. Donors, recipient organizations and vehicles are modeled as system data. The OpenStreetMap tile service is an external system that supplies map tiles for the network map (UC-17).

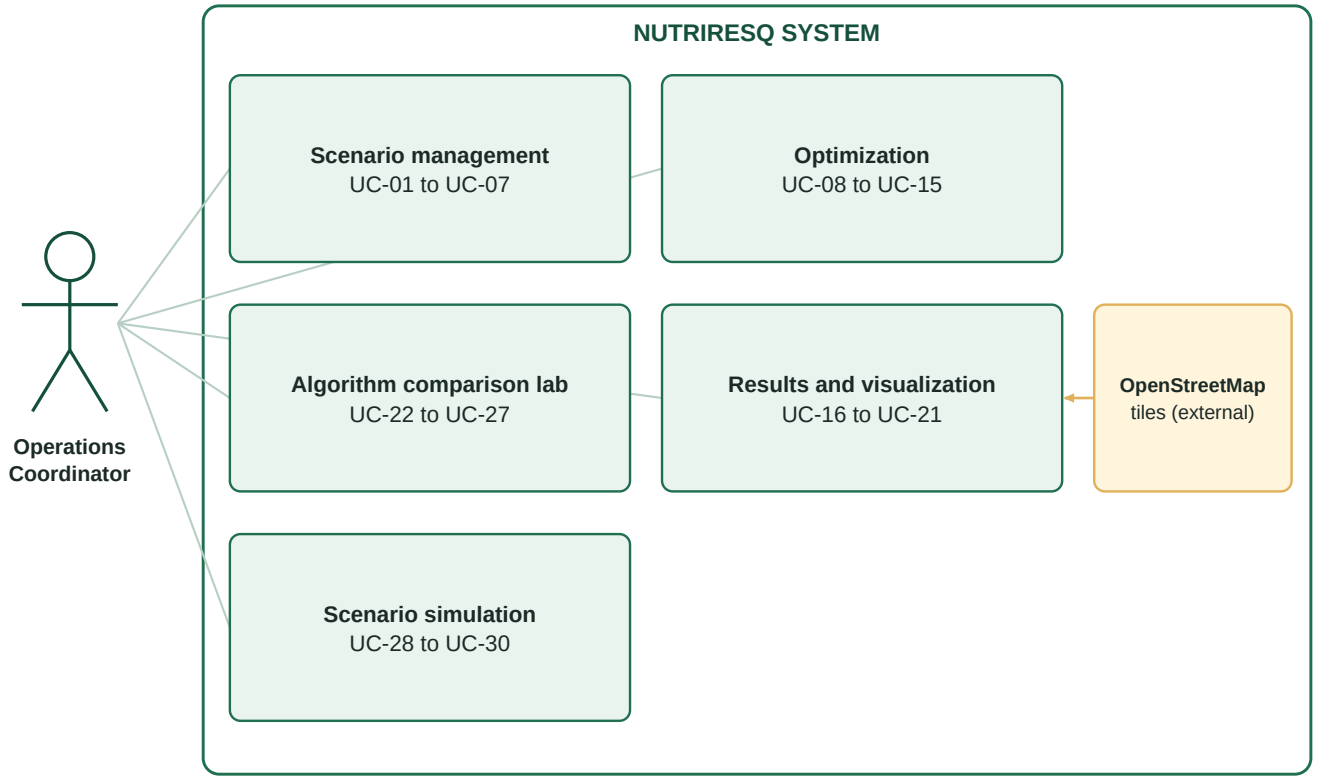


Figure 13.1 — Use case overview grouped by functional area

Scenario management

ID	Use case	Notes
UC-01	Create Scenario	Includes edit (UC-02) and view saved scenarios (UC-03)
UC-02	Edit Scenario Inputs	
UC-03	View Saved Scenarios	
UC-04	Manage Road Availability	
UC-05	Manage Food Supply and Eligibility	
UC-06	Manage Vehicle Availability and Capacity	
UC-07	Inspect Donors and Recipient Locations	

Optimization

ID	Use case	Notes
UC-08	Validate Scenario	Included by UC-09
UC-09	Run Integrated Optimization	Includes UC-10 to UC-15
UC-10	Calculate A* Paths	
UC-11	Analyze Closest Pair	
UC-12	Allocate Food using DP	
UC-13	Optimize Stops using ACO	
UC-14	Calculate Metrics	
UC-15	Save Optimization Run	

Results and visualization

ID	Use case	Notes
UC-16	View Optimization Dashboard	
UC-17	View Interactive Network Map	External: OpenStreetMap tile service
UC-18	Inspect Vehicle Allocations	
UC-19	Inspect Routes and Unreachable Stops	
UC-20	View Runtime and Fleet Utilization	
UC-21	View Previous Optimization Runs	

Algorithm comparison laboratory

ID	Use case	Notes
UC-22	Run Algorithm Benchmarks	Includes UC-23 to UC-26
UC-23	Compare A* vs Dijkstra	
UC-24	Compare DP vs Greedy	
UC-25	Compare Closest Pair vs Brute Force	
UC-26	Compare ACO vs Nearest Neighbor	
UC-27	View Benchmark Results	

Scenario simulation

ID	Use case	Notes
UC-28	Configure Simulation	Close roads; reduce vehicles; reduce food supply; change recipient set; mark food ineligible/expired
UC-29	Run Simulation	Includes validating the modified scenario, re-running affected algorithms and saving results
UC-30	Compare Before-and-After Results	

14 Project Structure

Keep the algorithm engine separate from the API and the frontend so the DAA implementations can be tested independently and explained in the report. This is a proposed structure, not something to create all at once; begin with the algorithm modules and their tests, then add the API and UI.

```
NutriResQ/
|-- client/                    React + TypeScript frontend
|   `-- src/
|       |-- components/        map/  charts/  tables/  common/
|       |-- pages/             Dashboard, NetworkExplorer, Allocation,
|       |                      RoutePlanner, AlgorithmLab, ScenarioSimulator
|       |-- api/  hooks/  types/  App.tsx
|-- server/                    Node.js + Express backend
|   `-- src/
|       |-- algorithms/        astar.js  multiKnapsack.js  closestPair.js  aco.js
|       |-- baselines/         dijkstra.js  greedyKnapsack.js
|       |                      bruteForceClosestPair.js  nearestNeighbor.js
|       |-- services/          optimizationService  scenarioService  benchmarkService
|       |-- routes/  controllers/  db/  validators/  app.js
|-- data/                      locations, roads, donations, recipients, vehicles (JSON)
|-- tests/                     algorithms/  integration/  scenarios/
|-- docs/                      problem-formulation, algorithm-design,
|                              complexity-analysis, test-results
|-- README.md  .gitignore
```

The architecture document additionally suggests backend folders for config, middleware, repositories, utils and migrations, and frontend folders for features and lib. These are responsibility boundaries, not permission to create dozens of empty files immediately; implement only what the current phase needs.

15 Testing and Evaluation Plan

NutriResQ must prove correctness, not just look convincing. Results are recorded from measurement; no percentage improvement is promised in advance.

Test type	What it checks
Correctness	A* returns valid paths; no closed road is traversed; every vehicle stays within capacity; no package is duplicated; route costs match the underlying road graph.
Cross-checks	A* cost vs Dijkstra; closest pair vs brute force; DP vs exhaustive enumeration on tiny multiple-knapsack instances; ACO vs nearest neighbor.
Performance	Increase locations, roads and packages; measure runtime (and memory where useful) under identical conditions for every compared method.
Failure	Disconnected networks, invalid inputs, closed roads, zero or unavailable vehicles, excess demand, oversized packages, expired donations and infeasible destinations.

For ACO use fixed seeds where possible and run multiple trials. For DP the small exhaustive comparison is especially valuable because it reveals errors in the multidimensional recurrence or reconstruction logic. A test runner is chosen and documented before Phase 2, and tests cover each algorithm, baselines, post-optimization invariants, API validation and errors, persistence and migrations, and at least one end-to-end scenario.

16 Mapping to the DAA CCP Requirements

Assignment requirement	NutriResQ implementation
Real-life problem formulation	Food rescue and redistribution optimization
Multiple algorithm paradigms	A*, multi-knapsack DP, Closest Pair, ACO
Shared underlying data	Locations, road network, food batches and vehicles
Disconnected-network handling	Report unreachable locations and infeasible routes
Tie-breaking	Deterministic rules for equal-cost paths and equal-valued choices
Road-failure extension	Close three roads; repair affected paths; compare with full recomputation
Multiple-knapsack constraints	Unique package assignment, capacity limits and backtracking
Over-demand case	Return the best feasible partial allocation
Oversized-item case	Exclude packages that cannot fit any vehicle
Integrated worked example	At least 10 locations and 20 roads, with allocation and route results
Sensitivity analysis	Remove the two highest-urgency recipient locations and rerun the pipeline
Complexity analysis	Derive time and space bounds for every implemented algorithm
Algorithm comparison	Compare against valid baseline algorithms
Edge-case tests	Disconnected graph, over-demand, oversized package, plus extra routing tests
Minimum six-page report	Report covering Parts A to C, adapted to NutriResQ

Note: The original brief describes an MST in Part A. NutriResQ instead uses A* for the greedy/search role because the instructor permits alternatives. Network analysis is therefore documented as shortest-path and reachability analysis, not as an MST or minimum spanning forest. The brief's incremental MST update requirement is MST-specific; the equivalent here is a road-failure experiment that invalidates three roads, repairs affected paths and compares that with full path recomputation. Do not claim it is an incremental MST algorithm. Because the multiple-knapsack requirement is unusually specific, give that algorithm particular attention in the implementation and the report.

17 Roadmap, Scope and Governance

17.1 Development roadmap

Build in this order. Do not spend most early time polishing the dashboard before the algorithms are correct: a minimal interface over a correct engine beats a beautiful interface over fabricated results.

Phase	Focus	Outcome
1 — Model and dataset	Define locations, roads, package weights/values, capacities, demands and road availability; create the synthetic dataset and document assumptions.	Documented model and seed data
2 — Algorithms	Build A*, multi-knapsack DP, Closest Pair and ACO as independent modules with unit tests and baselines.	Four tested modules plus baselines
3 — Integration	Connect pathfinding, allocation and routing; define how unreachable and infeasible cases are reported.	Deterministic optimization service
4 — Backend and database	Build the Express API; save scenarios, inputs, assignments, routes and benchmarks.	API plus PostgreSQL persistence
5 — Frontend	Build dashboard, map, allocation tables, comparison lab and scenario simulator on real API responses.	Interactive application
6 — CCP evaluation	Run all edge cases; produce the worked example; sensitivity analysis; derive complexity results.	Evidence for the report

Phase	Focus	Outcome
7 — Deployment and polish	Deploy frontend, API and database; prepare README, architecture diagram, screenshots and demo video.	Public, documented project

17.2 Scope boundaries

Build for the CCP	Consider as a future extension
Synthetic donor and recipient dataset	Live donor registrations
Four implemented algorithms	More advanced optimization techniques
Road availability and pathfinding	Live traffic and external routing integrations
Food allocation and capacity checks	Complex food-safety and cold-chain rules
Multi-stop routing	Real-time driver tracking
Comparison laboratory	Historical machine-learning predictions
Scenario simulator	Automatic donation notifications
PostgreSQL persistence	Multi-organization access controls
Deployment and documentation	Native mobile applications

17.3 Quality principles

- **Correctness:** every displayed allocation and route satisfies the documented constraints.
- **Transparency:** show inputs, assumptions, algorithm names, status, metrics and infeasibility warnings.
- **Reproducibility:** store the input snapshot, configuration, settings and results of each run.
- **Separation of concerns:** clear boundaries between UI, API, database access and algorithms.
- **No false claims:** ACO is a heuristic; OpenStreetMap tiles are not a routing engine; synthetic data is not live data.
- **Incremental development:** build a small, testable vertical slice before expanding.

17.4 Decisions still open before implementation

- the exact synthetic dataset and coordinate system;
- the numeric scale for food weights and package values in DP;
- the exact route semantics (which pickup and delivery stops are mandatory);
- the ACO parameters and reproducibility seed strategy;
- the authentication approach, if required for the first release;
- the final package versions and test framework; and
- the final visual design tokens.

Do not invent answers to these inside code. Record each decision in the relevant document before relying on it. If two documents conflict, stop and identify the conflict rather than silently choosing an interpretation.

17.5 Project documentation set

Document	What it governs
README.md	Project overview and documentation index
PRD.md	Product requirements and MVP acceptance criteria
ARCHITECTURE.md	Component boundaries and data flow
TECH_STACK.md	Approved stack and dependency rules
BACKEND_SCHEMA.md	PostgreSQL data model and persistence
DESIGN_SYSTEM.md	Visual design standards (planned)
CONTENT_SPEC.md	UI language and terminology (planned)
SECURITY.md	Security requirements (planned)
TEST_PLAN.md	Verification strategy (planned)
IMPLEMENTATION_PLAN.md	Phased build sequence (planned)
AGENTS.md	Instructions for AI coding agents (planned)

17.6 Definition of a credible demonstration

A reviewer should be able to change a road or vehicle, rerun the optimizer, inspect valid allocations and routes, see an unreachable or infeasible case handled honestly, compare at least one algorithm with its baseline, and review a saved run. The system should show how results were produced, not just present a polished dashboard.

17.7 Final summary

Core idea: an algorithm-driven decision-support platform that plans road paths, allocates surplus food to limited-capacity vehicles, analyzes geographic proximity and optimizes multi-stop delivery routes. **Chosen algorithms:** A* Search, Multiple Knapsack using Dynamic Programming, Closest Pair of Points using Divide and Conquer, and Ant Colony Optimization. The strongest version of NutriResQ lets a user change the data, run the optimizer, inspect feasible allocations and routes, compare algorithms, simulate disruptions and reproduce the results, delivering the academic substance of the DAA CCP together with a credible full-stack portfolio project.